Chương 2. Các mô hình dữ liệu

Mô hình dữ liệu quan hệ — Các dạng chuẩn hóa

Đỗ Thanh Hà

Khoa Trí tuệ nhân tạo
Học viện Công nghệ Bưu chính Viễn thông

Nội dung

- Các *bất thường* (anomalies) là gì và vì sao chúng xuất hiện.
- *Phân rã* một quan hệ: định nghĩa, lợi ích và hai cái giá phải trả.
- Hai tiêu chuẩn đánh giá một phép phân rã: *kết nối không mất mát* và *bảo toàn phụ thuộc*.
- Các *dạng chuẩn*: 1NF, 2NF, 3NF, BCNF.
- *Thuật toán* phân rã về BCNF và tổng hợp về 3NF.
- Vì sao đôi khi phải *dừng ở 3NF* thay vì đi tiếp lên BCNF.

Công cụ cần có từ chương trước

Toàn bộ chương này chạy trên ba thứ đã học ở bài phụ thuộc hàm: **bao đóng** $\{X\}^+$, **khóa ứng viên**, và **phủ tối thiểu**. Chuẩn hóa chỉ là *cách dùng* ba công cụ đó để quyết định tách bảng.

Nhắc nhanh: ba công cụ sẽ dùng liên tục

1. Bao đóng $\{X\}^+$
Tập mọi thuộc tính suy diễn được từ X. Dùng để trả lời *một* câu hỏi:

$$X \text{ là siêu khóa } \iff \{X\}^+ = R$$

2. Khóa ứng viên
Siêu khóa tối tiểu. Thuộc tính nằm trong *một khóa nào đó* gọi là **thuộc tính khóa** (prime); còn lại là **không khóa**.

3. Phủ tối thiểu
Tập FD nhỏ nhất tương đương F: vế phải một thuộc tính, vế trái không thừa, không FD nào thừa. Là đầu vào của thuật toán 3NF.

Một mẹo dùng suốt bài

Thuộc tính **không xuất hiện ở vế phải** của bất kỳ FD nào *bắt buộc* có mặt trong mọi khóa. Đây là bước đầu tiên khi tìm khóa — bỏ qua nó là nguồn sai sót phổ biến nhất.

Giới thiệu về các bất thường (*Anomalies*)

- Việc lựa chọn lược đồ quan hệ một cách **bất cẩn** có thể dẫn đến **dư thừa dữ liệu** (*redundancy*) và các bất thường liên quan.
- Các vấn đề xảy ra khi ta cố *nhồi nhét* quá nhiều thông tin vào một quan hệ duy nhất được gọi chung là **các bất thường**.

Gốc rễ của mọi bất thường

Một bảng lưu *một sự kiện ở nhiều dòng*. Khi sự kiện đó thay đổi, CSDL buộc phải sửa nhiều dòng cùng lúc — và không có gì bảo đảm điều đó xảy ra. Chuẩn hóa chính là kỹ thuật đưa mỗi sự kiện về đúng **một** chỗ.

Các loại bất thường

title	year	length	genre	studioName	starName
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With The Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

- **Dư thừa:** thời lượng, thể loại, studio của một phim bị lặp cho *mỗi* diễn viên.
- **Cập nhật:** sửa `length` của Star Wars thành 125 ở dòng 1 mà quên dòng 2–3 $\Rightarrow$ một phim có hai thời lượng.
- **Xóa:** xóa Vivien Leigh là mất luôn mọi thông tin về *Gone with the Wind* — đó là dòng cuối cùng còn giữ thông tin phim này.

Nguyên nhân chung: khóa là $\{\texttt{title}, \texttt{year}, \texttt{starName}\}$, nhưng $\{\texttt{title}, \texttt{year}\}$ đã xác định được `length`, `genre`, `studioName` — vế trái *nhỏ hơn* khóa.

Phân rã các quan hệ

- Phương pháp được chấp nhận để loại bỏ các bất thường là **phân rã** các quan hệ.
- **Định nghĩa.** Cho quan hệ $R(A_1, \ldots, A_n)$, ta nói R được phân rã thành $S(B_1, \ldots, B_m)$ và $T(C_1, \ldots, C_k)$ nếu:
  - $\{A_1, \ldots, A_n\} = \{B_1, \ldots, B_m\} \cup \{C_1, \ldots, C_k\}$
  - $S = \pi_{B_1,\ldots,B_m}(R)$ và $T = \pi_{C_1,\ldots,C_k}(R)$

Chú ý về điều kiện giao nhau

Định nghĩa *không* đòi hỏi hai tập thuộc tính rời nhau — ngược lại, chúng **phải** giao nhau thì mới ghép lại được bằng phép kết nối tự nhiên. Phần giao chính là thứ quyết định phân rã có mất mát hay không (slide sau).

Ví dụ: phân rã quan hệ

Quan hệ gốc $R(T, Y, L, G, S, N)$

title	year	length	genre	studioName	starName
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With The Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

$\downarrow$ tách theo $\{T, Y\} \rightarrow L, G, S$

Movie$(\underline{T}, \underline{Y}, L, G, S)$

title	year	len	genre	studio
Star Wars	1977	124	SciFi	Fox
Gone With The Wind	1939	231	drama	MGM
Wayne's World	1992	95	comedy	Paramount

StarsIn$(\underline{T}, \underline{Y}, \underline{N})$

title	year	starName
Star Wars	1977	Carrie Fisher
Star Wars	1977	Mark Hamill
Star Wars	1977	Harrison Ford
Gone With The Wind	1939	Vivien Leigh
Wayne's World	1992	Dana Carvey
Wayne's World	1992	Mike Meyers

Thảo luận: phân rã đã giải quyết được gì

- **Dư thừa đã mất:** thời lượng mỗi phim chỉ còn xuất hiện *một* lần.
- **Bất thường cập nhật đã mất:** đổi thời lượng Star Wars chỉ cần sửa *một* dòng của `Movie`. Không còn khả năng để lại hai giá trị mâu thuẫn.
- **Bất thường xóa đã mất:** xóa hết diễn viên của *Gone with the Wind* chỉ làm trống bảng `StarsIn`; thông tin phim vẫn còn trong `Movie`.

Tuy nhiên ...

- Truy vấn "phim nào Mark Hamill đóng và dài bao nhiêu phút" nay cần một phép **kết nối**. Chuẩn hóa đánh đổi *chi phí ghi* lấy *chi phí đọc*.
- Và quan trọng hơn: không phải phép phân rã nào cũng ghép lại được. Hai slide tiếp theo chỉ ra điều đó.

Phân rã: hai cái giá phải trả

- Ưu điểm của phân rã: loại bỏ dư thừa và các bất thường đi kèm.
- Nhưng một phép phân rã tùy tiện có thể mắc **một trong hai lỗi**:
  - **Mất mát khi kết nối** (*lossy join*) — ghép hai bảng con lại *không* cho lại quan hệ gốc.
  - **Mất phụ thuộc hàm** (*dependency loss*) — ghép lại thì đúng, nhưng không còn bảng con nào *kiểm tra* được một FD ban đầu, nên hệ quản trị không thể ép ràng buộc đó nữa.

Hai tiêu chuẩn của một phép phân rã tốt

(1) Kết nối không mất mát — bắt buộc, không bao giờ được hy sinh.
(2) Bảo toàn phụ thuộc — rất nên có, nhưng đôi khi không thể đạt cùng lúc với BCNF.

Ví dụ: phân rã mất mát (lossy join)

R **(gốc)**

A	B	C
1	2	3
4	2	5

$R_1 = \pi_{AB}(R)$

A	B
1	2
4	2

$R_2 = \pi_{BC}(R)$

B	C
2	3
2	5

$R_1 \bowtie R_2$

A	B	C
1	2	3
1	2	5
4	2	3
4	2	5

- Hai bộ tô đậm là **bộ giả** (*spurious tuples*): chúng không có trong R nhưng xuất hiện sau khi kết nối.
- Luôn luôn $R \subseteq R_1 \bowtie R_2$. Vì vậy "mất mát" **không phải** mất bộ — mà là **mất thông tin**: sau khi tách, CSDL không còn biết $A = 1$ đi với $C = 3$ hay $C = 5$. Kết quả là một quan hệ *nhiều* bộ hơn nhưng *nghèo* thông tin hơn.

Tiêu chuẩn kết nối không mất mát

Định lý (kiểm tra cho phân rã thành *hai* quan hệ)

Phân rã R thành R_1 và R_2 là **không mất mát** đối với F khi và chỉ khi F suy diễn được ít nhất một trong hai FD:

$$R_1 \cap R_2 \to R_1 \qquad \text{hoặc} \qquad R_1 \cap R_2 \to R_2$$

Nói cách khác: **phần giao phải là siêu khóa của ít nhất một bảng con.**

Áp dụng: $R_1 = AB$, $R_2 = BC$, phần giao $= \{B\}$. Không có FD nào trong bài, nên $\{B\}^+ = \{B\}$: B không xác định được A, cũng không xác định được C. Cả hai điều kiện đều hỏng $\Rightarrow$ phân rã mất mát. ✓

Thuật toán BCNF luôn an toàn

Thuật toán BCNF tách theo FD vi phạm $X \to Y$ thành $R_1 = \{X\}^+$ và $R_2 = X \cup (R \setminus \{X\}^+)$. Khi đó $R_1 \cap R_2 = X$ và theo đúng định nghĩa $X \to \{X\}^+ = R_1$. Điều kiện trên *tự động* thỏa mãn — đó là lý do phân rã BCNF **không bao giờ** mất mát.

Ví dụ: mất phụ thuộc hàm

Xét $R(A, B, C)$ với $F = \{A \to B,\ B \to C\}$, phân rã thành $R_1(A, B)$ và $R_2(A, C)$.

- **Không mất mát?** $R_1 \cap R_2 = \{A\}$, và $A \to B$ nên $A \to R_1$. **đạt** — ghép lại cho đúng R.
- **Các FD chiếu được:** $\pi_{AB}(F) = \{A \to B\}, \quad \pi_{AC}(F) = \{A \to C\}$.
- **Hợp lại:** $\{A \to B,\ A \to C\}$. Từ đây tính $\{B\}^+ = \{B\}$ — **không suy ra được** $B \to C$.

Hệ quả

Ràng buộc $B \to C$ *vẫn phải* đúng về mặt nghiệp vụ, nhưng không bảng con nào chứa đồng thời B và C. Muốn kiểm tra nó, hệ quản trị phải **kết nối hai bảng ở mỗi lần ghi** — quá đắt, nên trên thực tế ràng buộc đó bị bỏ, và dữ liệu sai có thể lọt vào CSDL mà không ai phát hiện.

Phân rã đúng ở đây là $R_1(A, B)$ và $R_2(B, C)$: giao $= \{B\}$, $B \to C$ nên không mất mát, và cả hai FD đều kiểm tra được cục bộ.

Các dạng chuẩn (Normal Forms)



Quan hệ bao hàm: BCNF $\subset$ 3NF $\subset$ 2NF $\subset$ 1NF. Mọi quan hệ BCNF đều là 3NF, nhưng *chiều ngược lại không đúng* — khoảng trống giữa 3NF và BCNF là nội dung chính của nửa sau bài này.

Ví dụ: hóa đơn

Acme Industries — HÓA ĐƠN (INVOICE)

Customer Number: 1454837
Customer: W. Coyote
General Delivery
Falling Rocks, AZ 84211
(599) 555-9345

Terms: Net 30
Ship Via: USPS

Order Date: 12/01/2008

Product No.	Description	Qty	Unit Price	Amount
SPR-2290	Super Strength Springs	2	24.00	$48.00
STR-67	Foot Straps, leather	2	2.50	$5.00
HLM-45	Deluxe Crash Helmet	1	67.88	$67.88
SFR-1	Rocket, solid fuel	1	128,200.40	$128,200.40
ELT-1	Emergency Location Transmitter	1	79.88	"FREE GIFT"
			TOTAL ORDER AMOUNT:	**$128,321.28**

Đây là *giao diện*, không phải lược đồ. Câu hỏi của chúng ta: nếu lưu toàn bộ hóa đơn này vào **một bảng duy nhất** thì bảng đó hỏng ở đâu?

Lưu vào một bảng duy nhất

Invoice ID	Customer Number	Customer	Terms	Ship via	Order date	Product No.	Description	Qty	Unit Price
1	1454837	W.Coyote, ...	N30	USPS	12/01/08	SPR-2290	Super Springs	2	24.00
1	1454837	W.Coyote, ...	N30	USPS	12/01/08	STR-67	Foot Straps	2	2.50
1	1454837	W.Coyote, ...	N30	USPS	12/01/08	HLM-45	Deluxe Helmet	1	67.88
1	1454837	W.Coyote, ...	N30	USPS	12/01/08	SFR-1	Rocket, Solid fuel	1	128,200.40
1	1454837	W.Coyote, ...	N30	USPS	12/01/08	ELT-1	Location Transmitter	1	79.88

Sáu cột đầu lặp lại y hệt ở cả năm dòng - dấu hiệu của dư thừa. Nhưng "nhìn thấy lặp" chưa đủ để kết luận; ta cần phát biểu các FD rồi đối chiếu với khóa.

FD và khóa của quan hệ INVOICE

Các FD theo ngữ nghĩa nghiệp vụ:

- Customer Number $\rightarrow$ Cname, Address
- Product No. $\rightarrow$ Description, Unit Price
- Invoice ID $\rightarrow$ Customer Number, Cname, Address, Terms, Ship Via, Order Date
- Invoice ID, Product No. $\rightarrow$ Quantity, Extended Amount

Khóa ứng viên:

- *Invoice ID* và *Product No.* không xuất hiện ở vế phải của FD nào $\Rightarrow$ cả hai phải nằm trong mọi khóa.
- $\{\text{Invoice ID}, \text{Product No.}\}^+ =$ toàn bộ thuộc tính $\Rightarrow$ đây là siêu khóa, và vì mọi khóa phải chứa nó nên nó là **khóa ứng viên duy nhất**.

Suy ra:

Ba FD đầu có vế trái là *tập con thực sự* của khóa, hoặc không liên quan đến khóa. Đó chính xác là những gì 2NF và 3NF sẽ lần lượt loại bỏ.

Dạng chuẩn thứ nhất (1NF)

- Quan hệ R ở **1NF** khi và chỉ khi:
  - Mọi miền giá trị chỉ chứa các giá trị **nguyên tử** (atomic);
  - Mỗi ô chứa **đúng một** giá trị từ miền đó — không danh sách, không nhóm lặp, không cấu trúc lồng nhau.

"Nguyên tử" phụ thuộc vào nghiệp vụ, không phải vào kiểu dữ liệu

Một chuỗi `"Hà Nội, Việt Nam"` là nguyên tử nếu ứng dụng không bao giờ tách nó ra; là *không* nguyên tử nếu có truy vấn "tìm mọi khách ở Việt Nam". Vì vậy 1NF là dạng chuẩn duy nhất không quyết định được chỉ bằng FD mà phải hỏi người dùng.

Lưu ý thực tế: các kiểu `JSON`, `ARRAY` của PostgreSQL/MySQL hiện đại phá vỡ 1NF một cách có chủ ý. Chúng hữu ích, nhưng mọi bất thường trong bài này sẽ quay lại — đó là đánh đổi có ý thức, không phải thiết kế mặc định.

1NF - trường hợp thuộc tính phức hợp

Không 1NF

Họ tên đầy đủ	
Tên	**Họ**
Peter	Smith
Laura	Veirs

Một thuộc tính có cấu trúc con.

Hai cách sửa, đều hợp lệ

Tên	Họ
Peter	Smith
Laura	Veirs

(a) Tách thành hai thuộc tính đơn.

Họ tên đầy đủ
Peter Smith
Laura Veirs

(b) Coi cả chuỗi là một giá trị nguyên tử.

Chọn (a) nếu ứng dụng cần sắp xếp theo họ, lọc theo tên, hay hiển thị riêng từng phần. Chọn (b) nếu họ tên luôn được xử lý như một khối. **Không có đáp án đúng tuyệt đối** - chỉ có đáp án đúng với một tập truy vấn cụ thể.

1NF - trường hợp giá trị đa trị

Quan hệ KHÁCH HÀNG — không 1NF

Mã KH	Tên	Họ	Số điện thoại
123	Pooja	Singh	555-861-2025, 192-122-1111
456	San	Zhang	(555) 403-1659 Ext. 53; 182-929-2929
789	Usman	Doe	555-808-9633

Cách 1 - thêm cột (nên tránh)

Mã	Tên	SĐT 1	SĐT 2
123	Pooja	555-861-2025	192-122-1111
456	San	(555) 403-1659	182-929-2929
789	Usman	555-808-9633	—

Cách 2 - tách bảng (đúng)

Mã KH	Số điện thoại
123	555-861-2025
123	192-122-1111
456	(555) 403-1659 Ext. 53
456	182-929-2929
789	555-808-9633

Vì sao cách 1 tệ: nó *đạt* 1NF về mặt hình thức nhưng đặt ra một giới hạn cứng (khách thứ ba có ba số thì sao?), sinh ra `NULL`, và làm câu hỏi "ai dùng số 192-122-1111?" phải quét mọi cột. Cách 2 không có nhược điểm nào trong số đó.

Tách Customer trước khi đi tiếp

Thuộc tính `Customer` trong hóa đơn gộp cả tên lẫn địa chỉ. Tách ra thành `Cname` và `Address`, tập FD trở thành:

- Customer Number $\rightarrow$ Cname, Address
- Product No. $\rightarrow$ Description, Unit Price
- Invoice ID $\rightarrow$ Customer Number, Cname, Address, Terms, Ship Via, Order Date
- Invoice ID, Product No. $\rightarrow$ Quantity, Extended Amount

Trạng thái hiện tại

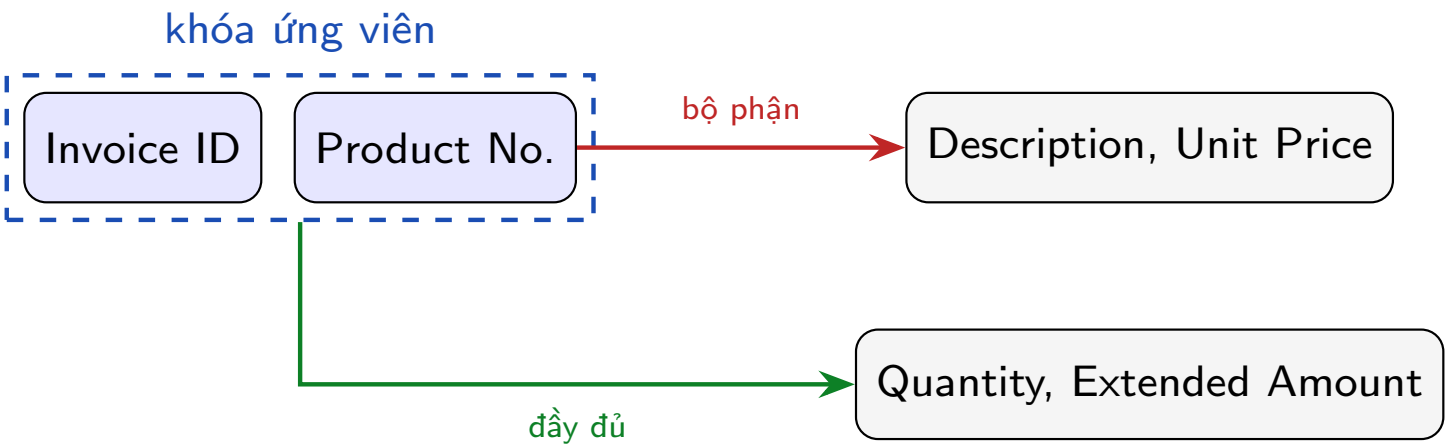
Quan hệ đã ở **1NF**. Khóa vẫn là {Invoice ID, Product No.}. Ba FD đầu đều có vế trái *không phải* là khóa — ta sẽ xử lý chúng theo thứ tự: 2NF lo các vế trái là *một phần* của khóa, 3NF lo các vế trái *nằm ngoài* khóa.

Dạng chuẩn thứ hai (2NF)

- Quan hệ R ở **2NF** khi và chỉ khi:
  - R ở **1NF**, VÀ
  - Mọi thuộc tính **không khóa** đều **phụ thuộc hàm đầy đủ** vào *mọi* khóa ứng viên - tức là không tồn tại **phụ thuộc bộ phận**.

Định nghĩa phụ thuộc bộ phận

$X \to A$ là **bộ phận** nếu X là tập con *thực sự* của một khóa ứng viên và A là thuộc tính không khóa. Nếu bỏ bất kỳ thuộc tính nào khỏi X mà FD không còn đúng thì $X \to A$ là **đầy đủ**.



Vì sao INVOICE vi phạm 2NF

Khóa ứng viên: {Invoice ID, Product No.}.

- Product No. $\rightarrow$ Description, Unit Price
  Vế trái là *một nửa* khóa $\Rightarrow$ phụ thuộc bộ phận. Hậu quả: mô tả và đơn giá của một sản phẩm bị lặp ở mọi hóa đơn có bán sản phẩm đó.
- Invoice ID $\rightarrow$ Customer Number, Cname, Address, Terms, Ship Via, Order Date
  Vế trái là *nửa kia* của khóa $\Rightarrow$ cũng là phụ thuộc bộ phận. Hậu quả: thông tin đầu hóa đơn bị lặp ở mọi dòng chi tiết.

Cách sửa cho mọi phụ thuộc bộ phận

Với mỗi phụ thuộc bộ phận $X \rightarrow Y$: đưa $X \cup Y$ ra một quan hệ mới với khóa X, và xóa Y khỏi quan hệ gốc (giữ lại X).

Hệ quả: nếu khóa ứng viên chỉ gồm **một** thuộc tính thì không thể có tập con thực sự khác rỗng - quan hệ *tự động* đạt 2NF.

Phân rã để đạt 2NF

Loại bỏ hai phụ thuộc bộ phận:

- Product No. $\rightarrow$ Description, Unit Price
- Invoice ID $\rightarrow$ Customer Number, Cname, Address, Terms, Ship Via, Order Date

---

Kết quả — cả ba đều đạt 2NF:

- **PRODUCT**(Product No., Description, Unit Price)
- **INVOICE**(Invoice ID, Customer Number, Cname, Address, Terms, Ship Via, Order Date)
- **INVOICE_ITEM**(Invoice ID, Product No., Quantity, Extended Amount)

Kiểm tra không mất mát. Tách PRODUCT ra: phần giao là {Product No.}, và Product No. là khóa của PRODUCT ✓. Tách INVOICE ra: phần giao là {Invoice ID}, là khóa của INVOICE ✓. Cả hai bước đều thỏa tiêu chuẩn ở slide trước, nên ghép lại cho đúng dữ liệu gốc.

Kết quả phân rã 2NF

INVOICE

Invoice ID	Cust. Number	CName	Address	Terms	Order Date
1	1454837	W.Coyote	General delivery, ...	Net 30	12/01/08

PRODUCT

Product No.	Description	Unit Price
SPR-2290	Super Strength Springs	24.00
STR-67	Foot Straps, leather	2.50
HLM-45	Deluxe Crash Helmet	67.88
SFR-1	Rocket, Solid fuel	128,200.40
ELT-1	Location Transmitter	79.88

INVOICE_ITEM

Inv.	Product No.	Qty	Amount
1	SPR-2290	2	$48.00
1	STR-67	2	$5.00
1	HLM-45	1	$67.88
1	SFR-1	1	$128,200.40
1	ELT-1	1	$0.00

Đơn giá sản phẩm nay chỉ lưu một lần. Nhưng bảng INVOICE vẫn còn vấn đề: `Cname` và `Address` phụ thuộc vào `Customer Number`, chứ không phải vào `Invoice ID`.

Dạng chuẩn thứ ba (3NF) - phát biểu cổ điển

- R ở **3NF** khi và chỉ khi R ở **2NF** và không thuộc tính không khóa nào **phụ thuộc bắc cầu** vào một khóa ứng viên.

Định nghĩa phụ thuộc bắc cầu — đủ cả bốn điều kiện

C phụ thuộc **bắc cầu** vào A nếu tồn tại B sao cho:

$$A \to B, \quad B \to C, \quad B \not\to A, \quad C \notin A \cup B$$

Hai điều kiện tô đỏ hay bị bỏ sót

Nếu thiếu $B \not\to A$: mọi quan hệ có **hai khóa ứng viên** K_1, K_2 đều có $K_1 \to K_2 \to$ (thuộc tính không khóa), và sẽ bị kết luận nhầm là vi phạm 3NF. Nhưng $K_2 \to K_1$ nên đây *không* phải bắc cầu.

Nếu thiếu $C \notin A \cup B$: $A \to B,\ B \to B$ cũng thành "bắc cầu".

3NF - phát biểu hiện đại (tương đương và dễ dùng hơn)

Định nghĩa dùng trong phần còn lại của bài

R ở **3NF** nếu với *mọi* FD không tầm thường $X \to A$ trong R:

X là **siêu khóa** của R **HOẶC** A là **thuộc tính khóa** (prime) của R.

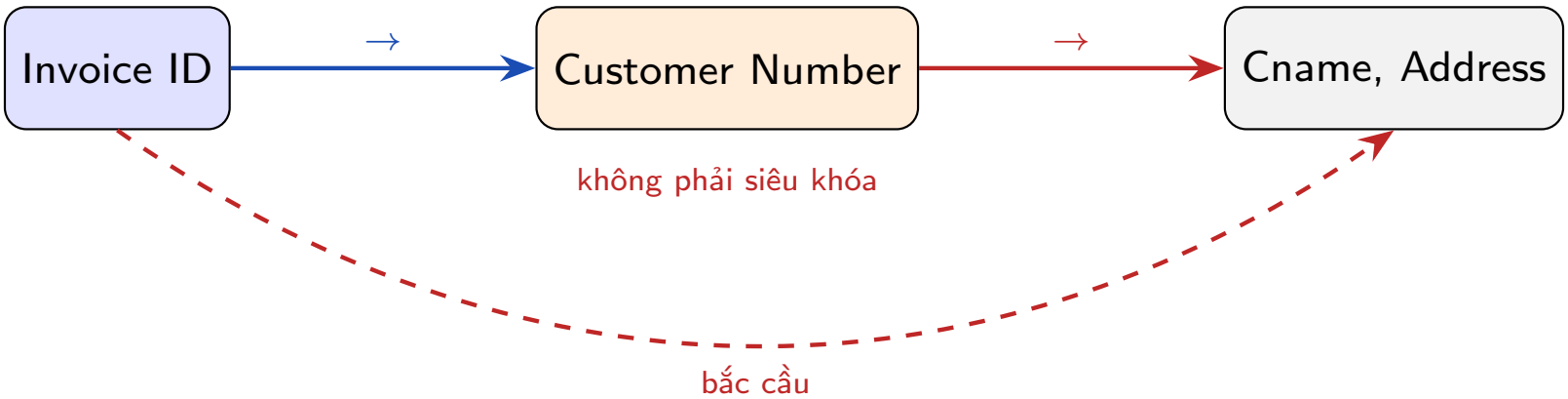
- Hai phát biểu **hoàn toàn tương đương**, nhưng phát biểu này kiểm tra được bằng máy: chỉ cần bao đóng và danh sách khóa.
- Nó cũng làm lộ rõ quan hệ với BCNF: BCNF là phát biểu trên sau khi **xóa vế "HOẶC A là thuộc tính khóa"**.

Quy trình kiểm tra 3NF

(1) Tìm *mọi* khóa ứng viên. **(2)** Liệt kê tập thuộc tính khóa. **(3)** Duyệt từng FD.
Chú ý: Bước (1) hay sai nhất - bỏ sót một khóa làm *tập prime nhỏ đi* và dẫn tới kết luận "vi phạm 3NF" hoàn toàn sai.

Vì sao INVOICE vi phạm 3NF

Xét quan hệ **INVOICE**(Invoice ID, Customer Number, Cname, Address, Terms, Ship Via, Order Date) sau bước 2NF. Khóa: {Invoice ID}.

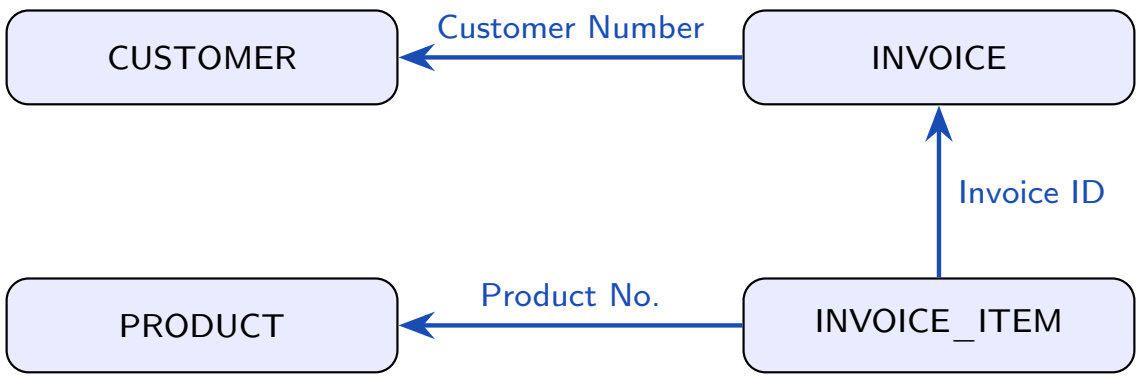


- Kiểm tra bốn điều kiện: InvID → CustNum ✓; CustNum → Cname ✓; CustNum ↛ InvID ✓(một khách có nhiều hóa đơn); Cname ∉ hai vế trái ✓. ⇒ bắc cầu thật.
- Bằng phát biểu hiện đại: CustNum → Cname, mà $\{\text{CustNum}\}^+ \neq R$ và Cname không là thuộc tính khóa ⇒ **vi phạm**.

Phân rã: tách **CUSTOMER**(Customer Number, Cname, Address), giữ Customer Number làm khóa ngoại trong INVOICE.

Lược đồ cuối cùng ở 3NF

- **CUSTOMER**(Customer Number, Cname, Address)
- **PRODUCT**(Product No., Description, Unit Price)
- **INVOICE**(Invoice ID, *Customer Number*, Terms, Ship Via, Order Date)
- **INVOICE_ITEM**(Invoice ID, Product No., Quantity, Extended Amount)



Mũi tên đi từ khóa ngoại tới khóa chính được tham chiếu (quy ước Elmasri). Cả bốn quan hệ đều ở 3NF; mỗi sự kiện nghiệp vụ — một khách hàng, một sản phẩm, một hóa đơn, một dòng hàng — nay được lưu đúng một chỗ.

Câu hỏi còn lại: 3NF đã đủ chưa? Slide tiếp theo cho thấy vẫn còn một loại dư thừa mà 3NF bỏ lọt.

Dạng chuẩn Boyce–Codd (BCNF)

Định nghĩa

Quan hệ R ở **BCNF** khi và chỉ khi: với *mọi* FD không tầm thường $A_1A_2\ldots A_n \to B_1B_2\ldots B_m$ đúng trên R, tập $\{A_1, \ldots, A_n\}$ phải là một **siêu khóa** của R.

- Nói gọn: **vế trái của mọi FD không tầm thường phải là siêu khóa**.
- So với 3NF, BCNF bỏ đi ngoại lệ dành cho thuộc tính khóa. Vì vậy

$$\text{BCNF} \Rightarrow \text{3NF} \quad \text{nhưng} \quad \text{3NF} \not\Rightarrow \text{BCNF}$$

Khi nào 3NF và BCNF khác nhau

Chỉ khác nhau khi quan hệ có các khóa ứng viên **chồng lấn** (giao nhau nhưng không trùng nhau). Nếu mọi khóa rời nhau, hoặc chỉ có một khóa, thì 3NF $\equiv$ BCNF.

Ví dụ: quan hệ phim không đạt BCNF

title	year	length	genre	studioName	starName
Star Wars	1977	124	SciFi	Fox	Carrie Fisher
Star Wars	1977	124	SciFi	Fox	Mark Hamill
Star Wars	1977	124	SciFi	Fox	Harrison Ford
Gone With The Wind	1939	231	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

- **Khóa:** $\{\texttt{title}, \texttt{year}, \texttt{starName}\}$ — cần cả `starName` vì một phim có nhiều diễn viên.
- **FD vi phạm:** $\{\texttt{title}, \texttt{year}\} \to \{\texttt{length}, \texttt{genre}, \texttt{studioName}\}$
- **Vì sao vi phạm:**
  $\{\texttt{title}, \texttt{year}\}^+ = \{\texttt{title}, \texttt{year}, \texttt{length}, \texttt{genre}, \texttt{studioName}\} \neq R$ — thiếu `starName`, nên vế trái **không** là siêu khóa. **vi phạm**
- Đây chính là dư thừa ta đã thấy ở đầu bài, nay được phát biểu bằng một tiêu chí *kiểm tra được*.

Hai mảnh sau khi phân rã - cả hai đều đạt BCNF

Movie(<u>title</u>, <u>year</u>, length, genre, studioName)

title	year	len	genre	studio
Star Wars	1977	124	SciFi	Fox
Gone With The Wind	1939	231	drama	MGM
Wayne's World	1992	95	comedy	Paramount

$\{\texttt{title}, \texttt{year}\}^+ =$ toàn bộ thuộc tính $\Rightarrow$ là siêu khóa. Mọi FD không tầm thường khác đều có vế trái chứa nó. **đạt**

StarsIn(<u>title</u>, <u>year</u>, <u>starName</u>)

title	year	starName
Star Wars	1977	Carrie Fisher
Star Wars	1977	Mark Hamill
Star Wars	1977	Harrison Ford
Gone With The Wind	1939	Vivien Leigh
Wayne's World	1992	Dana Carvey
Wayne's World	1992	Mike Meyers

Không có FD không tầm thường nào; khóa là cả ba thuộc tính. **đạt**

Kiểm tra không mất mát: phần giao là $\{\texttt{title}, \texttt{year}\}$, và $\{\texttt{title}, \texttt{year}\} \rightarrow$ Movie. Thỏa tiêu chuẩn ✓.

Chú ý: một quan hệ mà **mọi** thuộc tính đều nằm trong khóa duy nhất thì *luôn* ở BCNF - không có FD không tầm thường nào để vi phạm.

Ví dụ: $R(A, B, C, D, E)$

- $R = \{A, B, C, D, E\}$
- $F = \{\, A \to B, \quad BC \to E, \quad ED \to A \,\}$

Câu hỏi

1. Tìm **tất cả** khóa ứng viên của R.
2. R có đạt BCNF không?
3. R có đạt 3NF không?

Gợi ý: hãy bắt đầu bằng câu hỏi "thuộc tính nào không bao giờ xuất hiện ở vế phải?". Và hãy tìm *hết* khóa, đừng dừng lại ở khóa đầu tiên — câu 3 phụ thuộc hoàn toàn vào điều đó.

Lời giải - bước 1: tìm mọi khóa

Phân nhóm thuộc tính. Vế phải của F chứa $\{B, E, A\}$. Vậy:

- Nhóm **L** (chỉ ở vế trái): $\{C, D\}$ — **phải** có trong mọi khóa.
- Nhóm **M** (cả hai vế): $\{A, B, E\}$ — phải thử.

Xuất phát từ $\{C, D\}$: $\{C, D\}^+ = \{C, D\} \neq R$, nên phải bổ sung.

Tập thử	Bao đóng	Kết luận
$\{A, C, D\}$	$ACD \xrightarrow{A \to B} ABCD \xrightarrow{BC \to E} R$	**khóa**
$\{B, C, D\}$	$BCD \xrightarrow{BC \to E} BCDE \xrightarrow{ED \to A} R$	**khóa**
$\{C, D, E\}$	$CDE \xrightarrow{ED \to A} ACDE \xrightarrow{A \to B} R$	**khóa**

Cả ba đều tối tiểu vì $\{C, D\}^+ \neq R$. Không còn khả năng nào khác (mọi khóa phải chứa C, D và thêm đúng một trong A, B, E).

Kết quả

Khóa ứng viên: $\{A, C, D\}$, $\{B, C, D\}$, $\{C, D, E\}$.
Hợp lại: **mọi thuộc tính của R đều là thuộc tính khóa**. Tập thuộc tính không khóa là **rỗng**.

Lời giải - bước 2 và 3

Câu 2 — BCNF? Mọi vế trái phải là siêu khóa:

FD	Bao đóng vế trái	Siêu khóa?
$A \to B$	$\{A\}^+ = \{A, B\}$	không
$BC \to E$	$\{B, C\}^+ = \{B, C, E\}$	không
$ED \to A$	$\{E, D\}^+ = \{A, B, D, E\}$	không

$\Rightarrow$ R **vi phạm** BCNF (cả ba FD đều vi phạm).

Câu 3 — 3NF? Điều kiện: X là siêu khóa **HOẶC** vế phải là thuộc tính khóa.

FD	Vế phải có là thuộc tính khóa?	3NF
$A \to B$	$B \in \{B, C, D\}$ ✓	**đạt**
$BC \to E$	$E \in \{C, D, E\}$ ✓	**đạt**
$ED \to A$	$A \in \{A, C, D\}$ ✓	**đạt**

Kết luận

R **đạt 3NF** nhưng **không đạt BCNF** — đây chính là khoảng trống giữa hai dạng chuẩn. Nếu ở bước 1 bỏ sót khóa $\{B, C, D\}$ thì B bị xem là thuộc tính không khóa và câu 3 ra kết luận *ngược lại*.